When Code Becomes Cheap: Is AI Disrupting Open Source?

A mixed-methods case study on AI-generated code, technical debt, and security in open-source software

Safouan Bolbaroud
Master's Business Information Technology Management
University of Amsterdam
Supervisor: Chintan Amrit
6 February 2026

Table of Contents

1. The Issue 3
2. Positioning in the Academic Debate 4
   2.1. The Velocity-Quality trade-off 4
   2.2. Code review as the defence that gets overwhelmed 4
   2.3. AI-Generated code and known security weaknesses 5
   2.4. Technical Debt: The slow accumulation of unmaintainable code 5
   2.5. Detecting AI-Generated code 5
   2.6. The gap this thesis addresses 6
3. Research Questions 6
4. Methodology 7
   4.1. Research design 7
   4.2. Case selection strategy 7
   4.3. Phase 1: AI contribution detection (RQ1) 8
   4.4. Phase 2: Quantitative code and security analysis (RQ2 and RQ3) 8
   4.5. Phase 3: Qualitative evidence (RQ4) 8
5. Thesis outline and work plan 9
References 10

1. The Issue

Something changed in open-source software development around late 2022. The release of ChatGPT and the adoption of tools like GitHub Copilot and Cursor gave every developer, from seasoned kernel maintainers to first-time contributors, the ability to "write" code faster and in larger quantities than before. And up until recent this shift has gone unresearched. In November 2025, researchers at Carnegie Mellon University published a difference-in-differences study showing that the adoption of Cursor, a popular AI coding assistant, leads to a statistically significant and large increase in development velocity, but also a *persistent* increase in static analysis warnings and code complexity (He et al., 2025). A follow-up study in January 2026 confirmed this pattern for autonomous coding agents more broadly, finding that velocity gains are front-loaded and transient, while quality degradation endures (Agarwal, He & Vasilescu, 2026). In other words, AI makes code come faster, but the complexity it leaves behind slows projects down over time.

This would be a manageable problem if it only affected individual projects. But open-source software is not individual, it is infrastructure. According to Black Duck's 2025 report, 96% of all codebases contain open-source components, and 86% of risk-assessed codebases contain known open-source vulnerabilities (Black Duck, 2025). And what one must realize is that the consequences of quality degradation are immense, as mose companies and their software are dependent on thousands of open-source packages. Take the Log4Shell vulnerability in December 2021, which cost enterprises an average of €83,000 per incident, with 60% of affected organisations facing ransomware as a direct consequence (Sonatype, 2024). The XZ-Utils supply chain attack in 2024 showed that even a moderately popular package maintained by a single developer can become a nation-state attack vector (Sonatype, 2024). And in December 2025, React, the most widely used web framework, was hit by CVE-2025-55182, a CVSS 10.0 unauthenticated remote code execution vulnerability in its Server Components that was called "React2Shell" and actually affected even more hosts than Log4Shell (Intruder, 2025).

Yet despite the obvious intersection of these trends, no study has connected them. The CMU papers measure velocity and complexity but not security. The vulnerability literature documents incidents but does not investigate whether AI-generated contributions played a role. The code review literature establishes that review quality depends on time and expertise, but does not examine what happens when AI dramatically increases the volume of code that reviewers must absorb. And the emerging work on technical debt identification (Shivashankar et al., 2024) has not been applied to the specific context of AI-generated contributions.

This thesis addresses that gap. Rather than running another broad statistical survey across hundreds of repositories, which CMU has already done quite well, this study takes a different approach: a deep, mixed-methods case study of one to two mature open-source projects with recent, critical security incidents. The goal is not just to measure whether AI-generated code is present, but to find out whether it matters: for technical debt, for review quality, and ultimately for security. The study builds directly on the detection infrastructure developed by the CMU group (the AIDev dataset and associated methods), applies it to a security and technical debt analysis that they did not pursue, and enriches the picture with qualitative evidence from developer discourse. This combination of security focus, quantitative and qualitative methods

is what distinguishes this thesis from existing work and what makes it a meaningful contribution to the emerging academic debate on AI's real-world impact on software quality.

2. Positioning in the Academic Debate

2.1. The Velocity-Quality trade-off

The most directly relevant prior work comes from Vasilescu et al. at Carnegie Mellon University, whose papers have been accepted at the Mining Software Repositories conference (MSR 2026). He et al. (2025) presents the first causal study of AI coding tool adoption in open-source software. Using a difference-in-differences design with matched controls, they show that adopting Cursor leads to a significant but *transient* increase in development velocity (measured as commits and lines added), together with a significant and *persistent* increase in static analysis warnings and code complexity. They also showed that the quality degradation itself feeds back into velocity: the accumulated complexity becomes a drag on future development speed. The simplistic idea that "AI makes developers faster" is turned upside down by this discovery, which raises the possibility that the productivity increases are somewhat counterproductive. Agarwal, He, and Vasilescu (2026) extend this analysis by distinguishing between AI IDE assistants (like Cursor) and autonomous coding agents (like Devin and GitHub Copilot's agent mode). Using the AIDev dataset, a large-scale public dataset of nearly one million agent-authored pull requests across 116,000 repositories (Li et al., 2025), they find that velocity gains are strongest when an agent is the *first* AI tool in a project, and minimal where prior AI IDE usage already exists. Quality metrics, including static analysis warnings, cognitive complexity, duplication, and comment density, show similar degradation patterns regardless of whether the AI tool is an IDE assistant or an autonomous agent.

These papers establish two things that this thesis will take as its starting point. First, the velocity-quality trade-off from AI adoption is now documented and not just a matter of speculation. Second, the methodological infrastructure for detecting AI-generated code in open-source repositories exists and is publicly available. What these papers do *not* do is investigate whether the quality degradation they measure has security consequences, whether the rise in complexity represents the buildup of technical debt in an architectural sense, or the reasons for these patterns as perceived by the developers who encounter them.

2.2. Code review as the defence that gets overwhelmed

The empirical evidence on code review is well established and directly relevant. McIntosh et al. (2016) showed that carefully reviewed code components contain significantly fewer defects, and Bacchelli and Bird (2013) established that review effectiveness depends on reviewer expertise, available time, and attention. When reviewers are overwhelmed by volume or lack domain knowledge, defect detection rates drop. This creates a structural concern in the AI era: if AI tools dramatically increase the rate of code contribution, the review pipeline must absorb more code in the same time. In volunteer-driven open-source projects, where review capacity is already constrained, this could turn code review from a bottleneck into a failure point. The CMU papers document the speed increase, and thus this thesis investigates whether the review bottleneck has actual consequences for the security outcomes of specific projects.

2.3. AI-Generated code and known security weaknesses

Independent of the broad quality concerns measured by the CMU group, a separate line of research has documented that AI code generation tools produce code with measurable security vulnerabilities. Fu et al. (2023) found that approximately 29.5% of Python and 24.2% of JavaScript code generated by GitHub Copilot in real projects contained security weaknesses across 43 Common Weakness Enumeration (CWE) categories, including eight from the CWE Top-25. The most frequent was CWE-78, operating system command injection. Moonlight.io (2025) further showed that when Copilot itself is used to *review* code, it performs poorly at identifying security vulnerabilities, generating fewer than 20 comments across seven benchmark datasets containing hundreds of critical flaws. This creates a compounding risk: AI tools may simultaneously increase the volume of potentially vulnerable code while being unable to catch their own mistakes in review.

2.4. Technical Debt: The slow accumulation of unmaintainable code

Beyond immediate vulnerabilities, technical debt represents the accumulated cost of design shortcuts and deferred maintenance that make code harder to change and secure over time (Avgeriou et al., 2023). It manifests in measurable signals: code smells, high cyclomatic complexity, poor maintainability indices, duplication, and markers like TODO and FIXME comments. Zazworka et al. (2011) showed that high technical debt correlates with increased defect rates and reduced system changeability. More recently, Shivashankar et al. (2024) demonstrated that transformer-based models can classify technical debt types from GitHub issue data with high accuracy, suggesting that TD identification can now be partially automated.

AI models optimise for code that compiles and is valid in syntax, but they have no awareness of architecture, project-specific design principles, or long-term maintainability unless explicitly provided as context. The CMU papers measure complexity and static analysis warnings, which are *symptoms* of technical debt, but do not explicitly frame their findings in TD terms or investigate whether AI-generated code introduces specific, recognisable debt patterns (for example, code smells, architectural violations, deferred test coverage). This thesis aims to bridge that gap.

2.5. Detecting AI-Generated code

A key question is how do we properly detect AI-generated code? It seems like any code produced by a junior. Luckily there are datasets like the AIDev dataset (Li et al., 2025) which aggregates nearly one million pull requests authored by five AI coding agents (OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code) across 116,000+ repositories. In this dataset the detection relies on multiple signals: agent-specific commit metadata (for example, characteristic commit message patterns, bot account identifiers), IDE artefact traces (for example, the *.cursor/* directory), and neural network classifiers trained to distinguish AI-generated from human-written code based on stylistic features. This infrastructure means that the detection problem, which was the primary barrier to empirical research on AI's impact on real-world projects, is now somewhat solvable for agent-generated contributions. And this would allow us to research and detect AI more easily to further do the research on its effects.

2.6. The gap this thesis addresses

The above five parts of literature are mostly disconnected. The CMU group measures velocity and quality but not security or debt. The security literature documents AI code weaknesses in lab settings but not in real project outcomes. The code review literature explains why review bottlenecks matter but has not studied AI's role in creating them. The technical debt literature has not been applied to AI-generated contributions. And all existing work is purely quantitative; no study has gathered qualitative evidence from the developers who actually maintain these projects. This thesis aims to connect these through a deep case study, asking: in specific, mature open-source projects with documented security incidents (currently considering React.js as a primary case due to the severity and recency of CVE-2025-55182), can we trace whether AI-generated contributions played a role, whether technical debt accumulated differently in the AI era, and how the developers who maintain these projects experienced the shift?

The inspiration for this depth-over-breadth approach is the case study as done by Mockus, Fielding, and Herbsleb in 2002, whose in-depth analysis of just Apache and Mozilla in *ACM Transactions on Software Engineering and Methodology* became one of the most cited papers in software engineering. A small number of well-chosen, deeply analysed cases can produce insights that broad statistical surveys simply cannot.

3. Research Questions

The central research objective is to find out whether and how AI-generated code contributions have affected security outcomes, technical debt, and code quality in mature open-source projects. The overarching question is:

In mature open-source projects with documented security incidents, is there a traceable relationship between the rise of AI-generated code contributions and changes in technical debt, code review dynamics, and security outcomes?

This question is decomposed into four sub-questions, each building on the previous:

RQ1 (Detection): *To what extent are AI-generated contributions identifiable in the selected projects, and what is their prevalence over time?*

This foundational question determines what the study can see. Using the AIDev dataset and the detection methods from CMU, the study first maps the volume and timeline of AI-generated contributions in each selected case. The answer determines the confidence with which all subsequent findings can be interpreted. If AI contributions are sparse, the study adjusts its claims accordingly, essentially the strength of the conclusions really depend on how much evidence there is.

RQ2 (Technical Debt): *How have technical debt indicators and code quality metrics changed in the selected projects since the widespread availability of AI coding tools, and do AI-flagged contributions show different debt profiles than human-authored code?*

This question goes further than the static analysis warnings measured by the CMU papers by framing the analysis in technical debt terms. Metrics include cyclomatic complexity, maintainability index, code duplication, code smell density, and TODO/FIXME marker frequency, computed both at the project level (tracking overall trends pre- and post-2022) and at the contribution level (comparing AI-flagged versus human-authored code). This dual-level

analysis is important: a project-level trend could reflect many causes, but a contribution-level difference between AI and human code within the same project gives stronger evidence.

RQ3 (Security): *Is there a traceable link between AI-generated contributions and documented security vulnerabilities (CVEs) in the selected projects?*

This is the security question that no existing study has really asked with real project data. For each/the case, the study cross-references the project's CVE history with its commit history, identifying vulnerability-introducing commits and examining whether any are flagged as AI-generated. It also examines the review history of those commits: how long the review took, how many reviewers participated, and whether security concerns were raised. This question does not claim to establish causation from a small number of cases, but it aims to find out whether a possible link or correlation might exist and what the mechanism might be (for example, faster merges, less thorough review, specific weakness patterns).

RQ4 (Developer perspective): *How do project maintainers and contributors perceive the impact of AI-assisted contributions on code quality, review burden, and security?*

This qualitative question provides the explanatory depth that distinguishes this study from the purely quantitative CMU work. Evidence comes from two sources: mining public project communication (commit messages, issue trackers, pull request discussions, forum posts, and release notes) for discussions about AI-generated contributions, and where possible, interviews with project maintainers.

Together, these questions form a logical chain: RQ1 establishes what can be detected, RQ2 measures what changed in terms of code health, RQ3 investigates whether security was affected, and RQ4 explains why. Each question builds on the previous, and the combined picture will be a very rich overview and research that addresses a real gap.

4. Methodology

4.1. Research design

The study adopts a mixed-methods case study design focused on one to two mature, widely used open-source projects. This approach prioritises depth over breadth, with a combination of quantitative code analysis with qualitative, developer, insights that a large-scale statistical study cannot do.

4.2. Case selection strategy

Case selection is driven by four criteria. First, the project must be **mature and widely used**, meaning a long commit history (ideally spanning both the pre-AI and post-AI periods), active maintenance, and significant real-world deployment. Second, the project must have experienced **recent, documented security vulnerabilities** (CVEs), providing specific incidents that can anchor the security analysis. Third, the project should have **detectable AI contributions**, either through presence in the AIDev dataset or through other verifiable signals. Fourth, the project must have **rich public communication data** (issue trackers, pull request discussions, forums, release notes) to enable qualitative analysis.

Current projects that meet multiple criteria include React (CVE-2025-55182, with a CVSS of 10.0, the highest grade, in December 2025), Django (SQL injection vulnerability in 2024, long commit history, active discussion forums), and the Linux kernel (privilege escalation vulnerabilities in 2024). The final selection will be determined after verifying which projects

have sufficient AI-generated contributions detectable in their repositories. The study will proceed with the strongest candidate as the primary case, with a secondary case included for comparative analysis if feasible within the thesis timeline.

4.3. Phase 1: AI contribution detection (RQ1)

The first phase identifies and labels AI-generated contributions in each selected project. The primary detection strategy leverages the AIDev dataset (Li et al., 2025), which covers pull requests from five major coding agents across 116,000+ repositories. For selected projects that appear in AIDev, the labelled agent-authored PRs provide a ready-made starting point. This is supplemented by scanning for IDE agent traces (for example, *.cursor/* directories, characteristic commit patterns) and, if resources permit, applying or fine-tuning pre-trained code classifiers following the methods cited in the CMU work. The output is a labelled dataset of commits and pull requests classified by their origin, which serves as the foundation for all subsequent analysis.

4.4. Phase 2: Quantitative code and security analysis (RQ2 and RQ3)

The second phase computes code metrics at two levels. At the **project level**, key indicators are tracked over time across a pre-AI period (2018–2022) and a post-AI period (2023–2026): cyclomatic complexity, maintainability index, code duplication ratio, code smell density, and TODO/FIXME marker frequency. These are standard technical debt indicators computable with tools such as SonarQube, Radon, or equivalent static analysers. At the **contribution level**, the same metrics are compared between AI-flagged and human-authored contributions within the same project, controlling for project-specific confounds.

For the security dimension, the study traces vulnerability-introducing commits by cross-referencing CVE database entries with the project's commit history. Where vulnerability-introducing commits have been flagged as AI-generated, the study examines the full review trail: review duration, number of reviewers, reviewer comments, and whether security or quality concerns were raised. This commit-level tracing is the core analytical contribution, because it connects the abstract question of "does AI affect security" to specific, observable events in the project's history. Data sources include the GitHub Archive, static analysis tooling, and the National vulnerability database.

4.5. Phase 3: Qualitative evidence (RQ4)

The third phase gathers qualitative data to explain the patterns observed in Phase 2. Two strategies are pursued in parallel.

Public discourse mining. The primary qualitative data source is public project communication: commit messages, pull request review comments, issue tracker discussions, mailing lists (where applicable), and forum posts. Text search and AI-assisted categorisation tools are used to systematically identify developer discussions about AI-generated code, review workload, quality concerns, and policy decisions about AI contributions. This approach enables coverage of a large volume of discourse without depending on individual developers' willingness to participate.

Developer interviews. Where feasible, interviews with project maintainers and active contributors will supplement the discourse analysis. However, what is to be expected is a low

response rate from busy, volunteer open-source developers. Therefore the study does not depend on interview data for its core findings. The discourse mining approach should be enough and the interviews should only enrich the picture but are not a prerequisite for answering RQ4.

5. Thesis outline and work plan

Chapter 1: Introduction. Research context, motivation, and research questions.
Chapter 2: Literature Review. Comprehensive review covering code review effectiveness, AI code generation and security, technical debt theory, AI-code detection methods, and the case study tradition in empirical software engineering.
Chapter 3: Research Methodology. Case selection, AI detection approach, quantitative metrics and tools, qualitative data collection and analysis strategy, and validity considerations.
Chapter 4: Findings. Organised by research question: AI detection results, code quality and technical debt trends, security incident tracing, and developer discourse analysis.
Chapter 5: Discussion. Connecting everything across research questions, comparison with existing literature (particularly the CMU findings), implications for theory and practice, limitations, and threats to validity.
Chapter 6: Conclusion. Summary of contributions, practical recommendations for project maintainers and organisations managing OSS dependencies, and directions for future research.

Timeline

Period	*Activity*
Feb 2026	Deep review of CMU papers, AIDev dataset, and detection tooling, verify case selection criteria against available data
Mar–Apr 2026	Phase 1: Apply detection methods, build labelled contribution dataset for selected case(s), initial code metric computation
Apr–May 2026	Phase 2: Full quantitative analysis, technical debt indicators, CVE commit tracing, review trail analysis, start interview attempts for phase 3
May–Jun 2026	Phase 3: Discourse mining, interview attempts, qualitative analysis
Jun–Jul 2026	Connecting everything

References

Agarwal, S., He, H., & Vasilescu, B. (2026). AI IDEs or autonomous agents? Measuring the impact of coding agents on software development. arXiv preprint arXiv:2601.13597. DOI:10.48550/arXiv.2601.13597

Avgeriou, P., Kruchten, P., Ozkaya, I., & Seaman, C. (2023). Managing technical debt in software engineering. Dagstuhl Reports, 13(7), 152–173. DOI:10.4230/DagRep.6.4.110

Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In Proceedings of the 35th International Conference on Software Engineering (pp. 712–721). IEEE/ACM.

Black Duck. (2025). 2025 Open Source Security and Risk Analysis report. Synopsis. https://www.blackduck.com/blog/open-source-trends-ossra-report.html

He, H., Miller, C., Agarwal, S., Kästner, C., & Vasilescu, B. (2025). Does AI-assisted coding deliver? A difference-in-differences study of Cursor's impact on software projects. arXiv preprint arXiv:2511.04427. DOI:10.48550/arXiv.2310.02059

Intruder. (2025). The top vulnerabilities of 2025. https://www.intruder.io/blog/top-vulnerabilities-2025

Li, H., et al. (2025). The rise of AI teammates in software engineering (SE) 3.0. AIDev dataset. https://huggingface.co/datasets/hao-li/AIDev, Research: DOI:10.48550/arXiv.2507.15003

McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An empirical study of the impact of modern code review practices on software quality. Empirical Software Engineering, 21(5), 2146–2175. DOI:10.1007/s10664-015-9381-9

Mockus, A., Fielding, R. T., & Herbsleb, J. D. (2002). Two case studies of open source software development: Apache and Mozilla. ACM Transactions on Software Engineering and Methodology, 11(3), 309–346. DOI:10.1145/567793.567795

Moonlight.io. (2025). GitHub's Copilot Code Review: Can AI spot security flaws before you commit? https://www.themoonlight.io/en/review/githubs-copilot-code-review-can-ai-spot-security-flaws-before-you-commit

Fu Y., Liang P., Tahir A., Li Z., Shahin M., Yu J., Chen J. (2023). Security weaknesses of Copilot-generated code in GitHub projects: An empirical study. arXiv preprint arXiv:2310.02059. DOI:10.48550/arXiv.2310.02059

React Team. (2025). Critical security vulnerability in React Server Components. https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components

Shivashankar, K., Orucevic, M., Kruke, M. M., & Martini, A. (2024). Identifying technical debt and its types across diverse software projects issues. arXiv preprint arXiv:2408.09128. DOI:10.48550/arXiv.2408.09128

Sonatype. (2024). State of the software supply chain: 10 year look. https://www.sonatype.com/state-of-the-software-supply-chain/2024/10-year-look

Zazworka, N., Shaw, M. A., Shull, F., & Seaman, C. (2011). Investigating the impact of design debt on software quality. In Proceedings of the 2nd Workshop on Managing Technical Debt (pp. 17–23). ACM. DOI:10.1145/1985362.1985366